rdf-solve: Interoperable RDF for the Life Sciences through Automated Schema Mining and Integrated Semantic Mappings

Javier Millan Acosta Egon Willighagen More

March 31, 2026

Abstract

Biomedical knowledge is increasingly encoded in structured, web-accessible databases that can, in principle, be queried and combined automatically. The life sciences linked open data (LSLOD) cloud comprises hundreds of such databases expressed in the Resource Description Framework (RDF), yet each resource adopts its own vocabulary, class hierarchy, and modelling conventions. Because schemas are typically documented in publications rather than as machine-readable artefacts, discovering what classes and properties a dataset actually uses requires manual inspection and must be repeated whenever data are updated. Prior meta-analyses estimated vocabulary reuse across LSLOD at under 20 % and concluded the cloud is far less connected than commonly assumed, but these analyses remain descriptive: overlap is diagnosed statistically without constructing integration bridges or producing reusable schema artefacts. Here we present a methodology that automatically extracts approximate schemas from SPARQL endpoints, aligns them through a combination of instance-level URI probing and curated SSSOM mapping sets, and infers a connectivity graph revealing which dataset can be joined. Our approach materializes the extracted mapping-enriched schemas in linked-data standards including JSON-LD, SHACL, VoID, and LinkML, each carrying full provenance, making every artefact independently citable, and exportable to downstream formats such as SQL tables and Pydantic APIs. SSSOM and type-level mappings derived from shared instances from different namespaces expose hidden bridges between resources sharing no surface-level vocabulary. Inference over transitive, inverse, and generalised mapping chains

further extends connectivity beyond what any single mapping source provides. Applied to 235 LSLOD datasets, the pipeline produces citable, machine-readable schema artefacts and a navigable connectivity graph, lowering the barrier for federated querying, cross-resource data integration, and longitudinal schema monitoring. By making structural overlap explicit and actionable, this work lays a foundation for schema-aware query agents and systematic interoperability assessment across the biomedical semantic web. The methodology is released as an open-source Python package with a containerized web interface and reproducible pipeline scripts.

1 Introduction

1.1 Knowledge representation in the life sciences

Knowledge bases store structured, curated knowledge about a domain [?].

The Resource Description Framework (RDF) models information as subject–predicate–object triples where nodes are identified by Uniform Resource Identifiers (URIs).RDF Schema (RDFS) and the Web Ontology Language (OWL) layer class hierarchies, property constraints, and logical definitions on top of this model.The biomedical community was among the earliest adopters of Semantic Web technologies for knowledge management and clinical decision support, and the resulting Life Sciences Linked Open Data (LSLOD) cloud now spans hundreds of RDF resources [?].

RDF offers a uniform data model for heterogeneous domains. The use of URIs also makes resources web-dereferenceable: each entity can be looked up via standard HTTP. Semantics encoded in RDFS and OWL enable software reasoners to verify consistency and generate novel inferences from existing assertions , and the linking model exposes links to related resources even when underlying schemas differ .

1.2 The gap

Despite these properties, the landscape of semantic web resources in RDF for the life sciences faces significant interoperability challenges. There is semantic heterogeneity due to varying entity notations, diverse schemas, and insufficient common vocabulary reuse [?]. Analyzing complex RDF knowledge graphs requires a certain degree of computational skills and deep knowledge of domain-specific terminologies, and researchers often struggle to find sources available and accessible on the Web. Recent work on zero-shot SPARQL generation

shows that even LLM agents must probe endpoint schemas interactively before they can construct correct queries [?], which points at the practical importance of discoverable, machine-readable schema information. TogoMCP [?] reinforces this conclusion: it identifies concise, dynamically delivered schema context like ShEx shapes, sample triples, and dataset statistics packaged in a YAML descriptor as the most crucial component for accurate LLM-driven SPARQL generation, more valuable than complex behavioural orchestration logic. Moreover, generating correct queries for rare or unseen URIs (the out-of-vocabulary problem) remains difficult without a machine-readable schema to ground generation [?], which becomes an issue where RDF datasets attempt to reuse standard terms but end up representing them with different notations or namespaces, e.g. varying URI patterns for the same ChEBI identifier, which technically prevents automated integration [?].

Knowing which ontologies a dataset imports does not reveal its actual structure: which classes are instantiated, which entailments appear, how often a property appears, or where values are missing cannot be inferred from ontology declarations alone.

This is why two datasets that (partially) reuse the same ontology can have very different practical shapes while two datasets with disjoint vocabularies could have overlapping patterns that remain invisible without mappings. A meta-analysis of approximately 80 LSLOD sources estimated vocabulary reuse at under 20 % and concluded that the cloud "is not actually as densely connected as is often indicated" [?].

A further conceptual barrier stems from the evolution of schema philosophies. The Semantic Web community has traditionally treated schemas as *descriptive*, promoting flexibility, while the database community regards them as *prescriptive*, enforcing validation [?]. The development of RDF schema rule validation languages such as SHACL and ShEx Ahmetaj; Mihindukulasooriya [?, ?] reflects a convergence of these views.

1.3 Related work

Linked data profiling and RDF shape induction. Several profiling systems extract structural summaries from SPARQL endpoints. Loupe [?] mines class, property, and triple-pattern statistics and presents them through an interactive web interface. ABSTAT [?] computes ontology-driven linked data summaries using triples of the form subject type, property and object type, and applies a *minimalization* step that prunes patterns whose types are supertypes of more specific patterns already present, yielding more concise profiles than raw pattern enumeration. However, ABSTAT profiles are per-dataset and do not link patterns across resources or produce reusable schema artefacts in standard constraint

languages. ProLOD [?] profiles linked open data using statistical and structural analysis. SPORTAL [?] automatically generates VoID descriptions from public SPARQL endpoints, providing dataset-level metadata including class and property counts, but does not attempt schema-level pattern extraction or cross-endpoint mapping.

Several methods have been developed to analyze and characterize RDF data, each addressing a distinct slice of the schema discoverability problem. IndeGx [?] indexes dataset descriptions for federated source selection using declarative SPARQL rules and OWL RL reasoning to reflexively characterize endpoint quality and behaviour. VoID-ext [?] extends VoID with property and class partition statistics.

Tools to extract schemas automatically exist, including sheXer [?], VoID-generator [?], Loupe [?], Kamdar et al. [?], but adoption among life sciences dataset providers is slow. Mihindukulasooriya et al. [?] use machine learning (Random Forest classifiers trained on profiled RDF data from Loupe) to automatically induce integrity constraints in SHACL and ShEx, achieving up to 97 % precision in cardinality estimation. Automated extraction is also technically constrained by endpoint timeouts and deeply nested constraint patterns. VoID-ext service descriptions, where published, can enable features like autocompletion [?], demonstrating the practical value of machine-readable schema metadata. QLever extends this idea engine-side: it precomputes per-subject predicate patterns at index time to deliver context-sensitive SPARQL autocompletion with faster responses even over very large graphs [?], and more recently supports zero-shot SPARQL generation from natural language via an LLM agent that probes the endpoint schema interactively [?]. Still, most workflows, providers and endpoints do not explicitly share schema artifacts for downstream, decentralized uses.

LLM-driven query orchestration. TogoMCP [?] recasts the LLM as a protocol-driven orchestrator using the Model Context Protocol (MCP), separating entity resolution (via REST APIs such as UniProt and TogoID) from schema-guided SPARQL generation. Each database is described by a manually curated MIE (Metadata–Interoperability–Exchange) file, a concise YAML document containing ShEx shapes, sample triples, and statistics that is injected into the LLM context at query time. The approach demonstrates that structured schema context is the single most important factor for correct SPARQL generation, but inherits two limitations: MIE files must be created semi-automatically per database, requiring manual review, and the system operates on individual endpoints without native cross-endpoint JOIN support, relying on TogoID for sequential identifier conversion

between databases.

Complementary schema description approaches. Other frameworks describe the contents of RDF databases using alternative representations. The DBCLS-developed rdf-config [?] provides a human-readable YAML schema representation, manually curated for resources in the RDF Portal and used for finding links across the databases hosted in the RDF Portal. SPARQL query collections offer another useful complement for describing RDF resources, illustrating possible uses for a database.

More recently, Tirlet et al. [?] proposes treating SPARQL queries as composable modules that can be chained to build more complex analyses.

Achieving high performance for complex aggregate queries on large knowledge graphs remains an open challenge [?]. On the shape extraction side, sheXer [?] extracts ShEx shapes from RDF data, and shex-consolidator [?] merges independently extracted shape schemas into unified descriptions.

Exploration and Analytics of RDF. Sparklis [?] allows users to build analytical queries using natural language guidance directly against SPARQL endpoints. SynopsViz [?] provides hierarchical visual exploration and multilevel charting of large RDF datasets through incremental data retrieval and hierarchical aggregation [?].

Sparqloscope [?] generates targeted benchmarks from dataset statistics to identify engine-specific strengths and weaknesses across SPARQL 1.1 features, revealing for example that block-indexed engines handle `GROUP BY` with many groups far better than hash-based alternatives at billion-triple scale. Dense retrieval approaches adapt BERT-based re-ranking to RDF search by linearizing representative triples as model input [?], but scaling such methods to heterogeneous LSLOD endpoints remains an open problem.

Federated query planning and decentralized data access. Prior literature work reveals knowing *which endpoints hold which data* is a prerequisite for efficient federated execution regardless of the underlying access model. Federated SPARQL execution requires source selection: determining which federation members hold data relevant to each subquery. FedQPL [?] formalizes this as logical query plans over heterogeneous RDF interfaces. Link Traversal Query Processing (LTQP) [?] addresses this by using structural assumptions (Type Indexes) to prune the traversal space. LODsyndesis [?] takes a complementary approach: pre-computed specialized indexes and lattice-based connectivity metrics over hundreds of datasets enable global-scale knowledge services such as finding

the most connected datasets and measuring integration potential, but the system operates on static pre-computed indexes rather than live endpoint mining and does not produce reusable schema artefacts.

LSLOD meta-analysis. Kamdar et al. [**?**] conducted a comprehensive meta-analysis of the LSLOD cloud. Their Extraction Algorithm enumerated classes and the properties connecting them for a given endpoint. The extracted schema graph was used to analyze vocabulary reuse (estimated at 19.86 %), detect communities of similar content via GloVe word-embedding similarity and Louvain clustering, and quantify inter- and intra-linking patterns. Their finding that the LSLOD cloud is not actually as densely connected as is often indicated directly motivates the present work. However, their approach remains diagnostic: it identifies latent overlap via community detection but does not materialize integration bridges or produce actionable schema artefacts (other than the pickle NetworkX archived in figshare). The authors call for top-down approaches to monitor source accessibility and enforce vocabulary reuse standards [**?**].

Identifier normalization and knowledge graph assembly. Babel ORION [**?**] take a top-down approach to biomedical data integration: Babel normalizes identifiers across databases to a common namespace, and ORION assembles heterogeneous data into a unified knowledge graph conforming to the Biolink Model. This strategy yields a directly queryable integrated resource but requires community-maintained ontologies and curated transformation rules for each source, a design that trades automation for semantic precision. rdf-solve operates in the complementary bottom-up direction: rather than prescribing a target model, it discovers the *de facto* schemas that sources already expose and maps them post hoc, preserving each dataset's native representation.

Summary. Table **??** summarises the systems and methods discussed above, grouped by their primary contribution. The landscape spans dataset-level profiling (Loupe, ABSTAT, ProLOD, SPORTAL, VoID-generator), schema and shape extraction (sheXer, IndeGx, RDF Shape Induction), query-time schema discovery (GRASP, TogoMCP, Sparklis, Sparqloscope), federated planning and connectivity analysis (FedQPL, LODsyndesis), cloud-wide meta-analysis (Kamdar & Musen's Extraction Algorithm), and top-down integration (Babel ORION). No existing system combines fault-tolerant schema mining from live endpoints, multi-source mapping aggregation with inference expansion, and the materialization of cross-endpoint connectivity as reusable linked-data artefacts. rdf-solve is

designed to bridge that gap.

Table 1: Comparative overview of related work.

System	Authors	Goal	Access method	Year
ProLOD	Abedjan et al. [?].	Statistical and structural profiling of linked open data.	Data profiling toolset.	2014
SynopsViz	Bikakis et al. [?].	Hierarchical visual exploration and multilevel charting of large RDF datasets.	Incremental data retrieval and hierarchical aggregation.	2014
Loupe	Mihindukulasooriya et al. [?].	Class, property, and triple-pattern statistics for RDF datasets.	Interactive web interface over SPARQL endpoints.	2018
VoID-generator	Bolleman et al. [?].	Automated extraction of VoID-ext dataset descriptions.	Direct mining via SPARQL queries into VoID-ext catalogs of partitions.	??
ABSTAT	Spahiu et al. [?]	Ontology-driven linked data summaries with pattern minimization via subtype pruning.	Processes whole dataset; patterns reduced using ontology subtype graph.	2018
SPORTAL	Hasnain et al. [?]	Automated VoID descriptions from public SPARQL endpoints.	SPARQL queries against public endpoints.	2016
Sparklis	Ferré et al. [?]	Guided query building over SPARQL endpoints via controlled vocabulary.	Controlled-vocabulary interface against endpoints.	2017
LODsyndesis	Papadaki et al. [?]	Global-scale connectivity metrics and knowledge services across hundreds of datasets.	Pre-computed specialized indexes and lattice-based measurements.	2018
RDF Shape Induction	Mihindukulasooriya et al. [?]	Integrity constraint induction via ML (Random Forest) on profiled RDF data.	ML classifiers trained on Loupe statistical features.	2018
FedQPL	Cheng et al. [?]	Logical query plans over heterogeneous data access interfaces (TPF, SaGe, brTPF).	Formal query plan representation and source selection.	2020
Extraction Algorithm	Kamdar et al. [?]	LSLOD schema extraction, vocabulary reuse analysis, and community detection.	Automated SPARQL query templates.	2021
sheXer	Fernández-Álvarez et al. [?]	Automatic extraction of SHACL and ShEx shapes from RDF data.	Standalone shape extraction from dumps or endpoints.	2022
IndeGx	Maillot et al. [?]	Reflexive indexing of knowledge graphs to characterize endpoint quality and behaviour.	Declarative SPARQL rules and OWL RL reasoning.	2023
GRASP	Walter et al. [?]	Zero-shot SPARQL generation via LLM agent that probes endpoint schemas autonomously.	Interactive LLM function calls against QLever endpoints.	2025
Sparqloscope	Bast et al. [?]	Targeted benchmarks from dataset statistics for engine-specific performance profiling.	Python script using pre-computed statistics.	2025
TogoMCP	Kinjo et al. [?]	Schema-guided SPARQL generation via curated context files injected into LLM prompts.	YAML context files (MIE) employed by a Model Context Protocol.	2025

Continued on next page

Table ?? continued

System	Authors	Goal	Access method	Year
Babel / ORION	Carter et al. [?]	Identifier normalization (Babel) and model transformation to Biolink Model (ORION).	APIss.	2026

Several observations emerge from this comparison. First, most profiling tools (Loupe, ABSTAT, ProLOD, SPORTAL) produce per-dataset summaries but do not link findings across resources; rdf-solve's mapping layer and connectivity graph fill this gap. Second, the LLM-driven systems (GRASP, TogoMCP) demonstrate the value of machine-readable schema context for query generation, but operate on single endpoints at query time; the persistent, multi-format artefacts produced by rdf-solve could serve as upstream context for such agents across the entire LSLOD cloud. Third, prior cloud-wide analyses (Kamdar & Musen, LODsyndesis) quantify fragmentation or connectivity diagnostically, without materializing actionable integration bridges; rdf-solve produces typed mapping edges and a connectivity graph that answer *which endpoints can be joined*, not only *which schemas are similar*. Finally, top-down integration efforts (Babel ORION) achieve high semantic precision at the cost of per-source curation; rdf-solve's bottom-up, automated approach trades curation control for breadth of coverage and repeatability.

2 Methods

2.1 Scope

Formally, schema-based interoperability requires two capabilities. The first is measuring *consistency* within a dataset: how stable is a structural pattern across instances. Consistency scores, such as the coverage annotations introduced in sheXer Fernández-Álvarez et al. [?], capture how often a class–property–value-type pattern actually occurs. The second is measuring *alignment* across datasets: identifying classes or properties in different graphs that correspond to each other. This requires curated mappings or cross-references that link entities across resources.

Several tools address parts of this space in isolation; however, no existing system connects schema extraction, multi-format serialization, mapping inference, and cross-resource connectivity analysis to effectively expose links into a single pipeline.

rdf-solve addresses this gap. Building on the class-then-property paradigm of prior schema profiling work, it contributes:

1. When possible, local mining on QLever, which demands disk and RAM (Section **??**). Because a local QLever server imposes no network timeouts or rate limits, the pipeline first attempts a *one-shot* strategy: three unpaginated `SELECT DISTINCT` queries (typed-object, literal, untyped-URI) that exhaust the full local index without pagination or batching. The multi-level fallback chain (bisection, adaptive pagination, predicate-first decomposition) can be used as an alternative if any one-shot query fails. Remote endpoints, where no data dump is available, always use the fallback chain;

2. Fault-tolerant remote endpoint mining with multi-level fallbacks (bisection, adaptive pagination, predicate-first decomposition) that attempts to exhaust all possibilities before silently discarding classes on failure (Section **??**);

3. a configurable treatment of untyped URI patterns: the default strategy records them as bare predicates, while an alternative strategy (`-untyped-as-classes`) promotes the object URIs to candidate classes, enabling a second pass over the same indexed data that reflects a different modelling assumption (Section **??**);

4. a mapping layer that aggregates instance-based URI probing, curated SSSOM mapping sets, and SeMRA identifier mappings into a unified type-level correspondence graph, with inference expansion via transitivity, inversion, and generalisation (Section **??**);

5. serialization of every extraction step in standard linked data formats (JSON-LD, VoID, SHACL, LinkML) with full provenance metadata, including author attribution, timing, and tool version, so that each artefact is independently citable; and

6. a connectivity graph that identifies which endpoint pairs can be concretely joined via shared ontology classes and inferred mappings (Section **??**).

The tool is open-source, CLI-driven, and ships as a containerized web application.

2.2 Schema Mining

rdf-solve approximates the schema of a SPARQL endpoint by mining co-occurrence patterns from instance data, but minimizing timeout and service cost-related incompleteness of the mined results by prioritizing local mining and, if performingit on a remote endpoint:

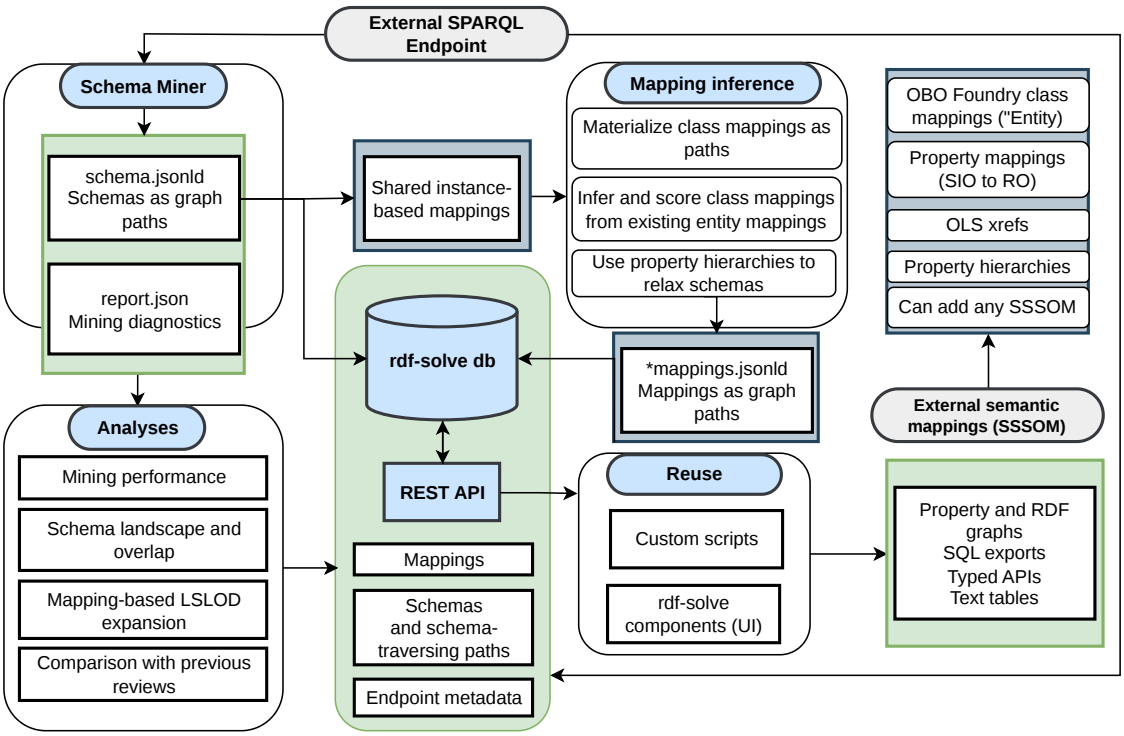

Figure 1: Workflow of the rdf-solve pipeline: from data source selection through schema mining, mapping aggregation, inference expansion, and connectivity graph construction.

(i) configurable class batching via SPARQL `VALUES` bindings. (ii) a multi-level fallback chain (batch bisection, pagination with adaptive page sizing, and predicate-first decomposition) that degrades gracefully instead of discarding classes on failure; and (iii) separation of pattern discovery from counting, ensuring that a count-query failure does not lose already-discovered patterns.

The extracted patterns of $(class_{subject}, p, class_{object})$ are closely related to those in related work [?] [?] [?] [?] [?] [?] [?]

ABSTAT [?] additionally applies a minimalization step that prunes patterns whose subject or object types are supertypes of more specific patterns already present, reducing profile size at the cost of discarding supertype-level information. Tools like VoID generator [?] or shexer [?] do not apply such pruning, retaining all observed patterns.

Likewise, rdf-solve retains all observed patterns. Crucially, all of the above remain per-dataset tools; rdf-solve feeds its patterns into a cross-resource mapping and connectivity layer that links findings across endpoints.

Three pattern types are recorded: typed-object patterns (c, p, o) where o is the `rdf:type` of the object URI; literal patterns (c, p, dt) where dt is the object datatype; and untyped-URI patterns (c, p) where no `rdf:type` was retrievable for the object under the current graph scope and inference presented by the endpoint.

The treatment of the third pattern type is configurable. In the default strategy, untyped-URI patterns are retained as bare predicate links whose object type is unknown. An alternative strategy promotes those object URIs to candidate classes, effectively treating the absence of an explicit `rdf:type` assertion as a modelling gap rather than an absence of class membership.

Both strategies can be applied to the same indexed data in consecutive passes, yielding two schema variants for the same source that can be compared or combined. This is particularly relevant for local mining (Section **??**), where re-running the algorithm over an already-indexed QLever instance adds negligible load.

Class and property labels are resolved from a priority-ordered label vocabulary: `rdfs:label` is preferred, followed by `skos:prefLabel`, `dc:title`, `IAO_0000118` (alternative term), and `skos:altLabel`.

Falling back to the local URI fragment is a last resort. The multi-vocabulary lookup is relevant for endpoints that use OBO-style annotation properties rather than RDFS labels.

Notation. $Q(B, G)$ denotes query template Q evaluated for class batch B restricted to the named graphs in G, a set of zero or more URIs supplied via `-graph-uri`. When $G = \varnothing$ no `GRAPH` clause is added and the query runs over the endpoint's default dataset. When $|G| = 1$ a single `GRAPH <uri> { ...}` clause wraps the pattern; when $|G| > 1$ a `VALUES (?g) { ...} GRAPH ?g { ...}` block iterates over each named graph. In all cases the scope is a query-level restriction; endpoint dataset configuration may override or ignore it. $\top$ / $\bot$ denote success / failure (timeout or cost-limit). $B_{[:m]}$ and $B_{[m:]}$ are the two halves of B after splitting at $m = \lfloor |B|/2 \rfloor$; bisecting reduces the number of classes in the `VALUES` binding, narrowing the class-filter scope and isolating expensive classes into separate sub-queries. $\ell_{\min}$: minimum page size (60 % of the initial `-chunk-size`); pagination is abandoned after three consecutive failed reductions at the same offset. Q_C: `SELECT DISTINCT ?c WHERE {?s a ?c}` (type enumeration). $Q_p(c)$: `SELECT DISTINCT ?p WHERE {?s a <c>; ?p ?o}` (predicate enumeration for class c). $Q^p_{\text{to}}(c, p)$: Q_{to} restricted to predicate p for class c. $t \models (s, p, o)$: triple $t = (s', p', o')$ matches pattern (s, p, o) operationally if the endpoint's evaluation of `?s a <s>; <p> ?o; ?o a <o>` (or the literal/datatype variant) binds t; this depends on the endpoint's graph scope, asserted `rdf:type` triples, and any active inference regime. $n_{(s,p,o)} = |\{t \mid t \models (s, p, o)\}|$ is the co-occurrence count as returned by the endpoint's `COUNT(*)` aggregate. Q^{cnt}_{to}, Q^{cnt}_{lit}, Q^{cnt}_{uri}: the corresponding `SELECT ...COUNT(*) ...GROUP BY` variants of Q_{to}, Q_{lit}, Q_{uri}; they accept the same `VALUES` batching and fallback chain.

Procedure. $\textsc{Query}(B, Q, E, G)$ sends $Q(B, G)$ to endpoint E as a single `SELECT DISTINCT` query. On $\top$ it returns the results immediately. On $\bot$, if $|B| > 1$ the batch is bisected and each half retried recursively: reducing the `VALUES` binding narrows the set of classes evaluated per sub-query, so that a class whose query exceeds the endpoint's cost limit does not cause the remaining classes in the same batch to be skipped.

For single-class batches ($|B| = 1$) the class filter is already minimal, so the fallback is pagination (`LIMIT` ℓ / `OFFSET` $k\ell$, $k = 0, 1, 2, \ldots$).

If a page times out, ℓ is reduced by a configurable shrink factor (default 0.85, i.e. a $\sim$15 % reduction per attempt) and the same page is retried after a cooldown pause; the chunk size is clamped to a floor of 60 % of the original value and at most three consecutive reductions are attempted per offset. If all reductions fail, pagination is abandoned and, for Q_{to}, a predicate-first decomposition is attempted: $Q_p(B[0])$ enumerates predicates P and $Q^p_{\text{to}}(B[0], p)$ is called once per $p \in P$; if Q_p itself fails, the class contributes no typed-object

patterns. Any results accumulated before pagination was abandoned are retained. If all strategies fail, QUERY returns $\emptyset$.

Phase 1 enumerates the asserted instantiated types C visible under G. When a `class_chunk_size` is configured, the type-enumeration query is paginated with `LIMIT`/`OFFSET` steps using the adaptive page-sizing mechanism (the page size is reduced on timeout, as described above); otherwise Q_C is sent as a single unpaginated query. Phase 1 does *not* use the bisect/decompose fallback chain: the type-enumeration query is a lightweight single-hop scan (`?s a ?class`) that does not involve the multi-way joins that motivate bisection in Phase 2. Phase 2 partitions C into batches B of configurable size and calls QUERY three times per batch (Q_{to}, Q_{lit}, and Q_{uri}) accumulating patterns into S. If $G \neq \varnothing$ and Phase 2 yields $S = \emptyset$ despite $C \neq \emptyset$, an *ontology-graph fallback* is triggered: Phase 2 is re-run with $G' = \varnothing$, because the named graph may contain only class definitions while instances reside in the default graph. Optionally (enabled by default, disabled with `-no-counts`), a counting phase follows: the unique subject classes in S are re-batched identically to Phase 2 and each batch is submitted as a grouped `SELECT ...COUNT(*) ...GROUP BY` query through QUERY, so count queries benefit from the same bisect and pagination fallbacks as pattern queries. If a count query fails for a particular class (after all fallbacks), the corresponding patterns are retained in S with no count.

Execution strategy selection and schema completeness. For local QLever endpoints the preferred execution path is the one-shot strategy: a single unpaginated `SELECT DISTINCT` issued before any fallback is attempted. If it succeeds, the schema is complete in one round-trip and the fallback chain is never entered. Only when the one-shot query fails does the pipeline activate the multi-level fallback chain (Algorithm **??**). Remote endpoints always begin with the fallback chain, as the endpoint's timeout and cost-limit behaviour is unknown.

When a source has been mined under multiple execution strategies (e.g. default and `untyped-as-classes`, or local and remote), the pipeline selects the most complete result for downstream analysis. Completeness is assessed by two criteria applied in order: first, whether the counting phase succeeded for all classes (i.e. all patterns carry co-occurrence counts rather than null); second, total pattern count. The run with full count coverage is preferred; among runs with equal coverage, the one with the larger pattern set is retained. This selection is recorded in the provenance block of the emitted schema, allowing the choice to be audited and overridden.

Algorithm ?? summarizes the full procedure.

Algorithm 1 Schema pattern mining

Require: endpoint E; graph scope G
Ensure: S (observed co-occurrence patterns); $\emptyset$ for a batch may mean failure or genuinely no patterns; incompleteness is logged but not encoded in S

$S \leftarrow \emptyset$
Phase 1 - class discovery
$C \leftarrow \textsc{Query}(\varnothing, Q_C, E, G)$ ▷ paginated with adaptive page sizing if `class_chunk_size` set; single unpaginated query otherwise
Phase 2 - pattern mining
for each batch $B \subseteq C$ **do**
  $S \leftarrow S \cup \textsc{Query}(B, Q_{\text{to}}, E, G)$ ▷ typed-object patterns
  $S \leftarrow S \cup \textsc{Query}(B, Q_{\text{lit}}, E, G)$ ▷ literal patterns
  $S \leftarrow S \cup \textsc{Query}(B, Q_{\text{uri}}, E, G)$ ▷ untyped-URI patterns
end for
if $S = \emptyset$ **and** $C \neq \emptyset$ **and** $G \neq \varnothing$ **then** ▷ ontology-graph fallback
  re-run Phase 2 with $G' = \varnothing$
end if
Counting phase (optional; skipped with `-no-counts`)
$N \leftarrow \emptyset$
for each batch $B \subseteq C_S$ **do** ▷ C_S: unique subject classes in S
  $N \leftarrow N \cup \textsc{Query}(B, Q^{cnt}_{\text{to}}, E, G)$ ▷ typed-object counts
  $N \leftarrow N \cup \textsc{Query}(B, Q^{cnt}_{\text{lit}}, E, G)$ ▷ literal counts
  $N \leftarrow N \cup \textsc{Query}(B, Q^{cnt}_{\text{uri}}, E, G)$ ▷ untyped-URI counts
end for
$S \leftarrow \{\, (s, p, o, n_{(s,p,o)}) \mid (s, p, o) \in S \,\}$ ▷ merge; n = `null` when no count retrieved
return S

function $\textsc{Query}(B, Q, E, G)$
  if $Q(B, G) = \top$ **then return** results
  end if ▷ single `SELECT DISTINCT`
  if $|B| > 1$ **then** ▷ bisect: isolates expensive classes into separate sub-queries
    return $\textsc{Query}(B_{[:m]}, Q, E, G) \cup \textsc{Query}(B_{[m:]}, Q, E, G)$
  end if
  $R \leftarrow \emptyset$; $k \leftarrow 0$; $r \leftarrow 0$ ▷ r: consecutive shrink attempts at this offset
  repeat ▷ paginate single-class result set (`SELECT DISTINCT`)
    if page $Q(B, G, \ell, k) = \top$ **then**
      $R \leftarrow R \cup$ page results; $k \leftarrow k + 1$; $r \leftarrow 0$
    else
      $\ell \leftarrow \max(\lfloor 0.85\,\ell \rfloor,\ \ell_{\min})$; $r \leftarrow r + 1$; **retry page** k
      if $r > 3$ **or** $\ell = \ell_{\min}$ unchanged **then goto** *decompose*
      end if
    end if
  until empty page received
  return R ▷ partial results are retained even if pagination is later abandoned

  decompose:
  if $Q = Q_{\text{to}}$ **then** ▷ predicate-first decomposition (last resort)
    $P \leftarrow \textsc{Query}(\{B[0]\}, Q_p(B[0]), E, G)$
    return $\bigcup_{p \in P} \textsc{Query}(\{B[0]\}, Q^p_{\text{to}}(B[0], p), E, G)$
  end if
  return $\emptyset$ ▷ strategies exhausted
end function

2.3 Semantic mapping layer

The mapping layer aggregates cross-resource correspondences from three complementary sources:

1. **Instance-based URI probing.** For each bioregistry [?] prefix (e.g. `ensembl`, `chebi`), all registered URI format prefixes are resolved via `bioregistry.get_uri_prefixes`. Each format is then tested against every data source via a `STRSTARTS`-based `SELECT DISTINCT ?c` query that retrieves the `rdf:type` classes whose instances carry URIs matching that prefix. Pairwise mapping edges are generated between every two *distinct* classes that both matched the resource, including two classes within the *same* dataset (e.g. `Gene` and `GeneAnnotation` in the same endpoint both having Ensembl instance URIs). The default predicate is `skos:narrowMatch`, reflecting that URI-prefix co-occurrence is necessary but not sufficient evidence for semantic equivalence.

2. **SSSOM imports.** Curated mapping sets published in the Simple Standard for Sharing Ontology Mappings (SSSOM) format [?] are downloaded and converted to the internal mapping representation. Each `.sssom.tsv` file is parsed including its YAML front-matter; predicate URIs are resolved first through the file's own `curie_map` and then through a built-in fallback table for standard SKOS and OWL predicates. SeMRA [?] provides systematically assembled identifier-level mappings covering prefix combinations (e.g. NCIT–ChEBI) not visible from endpoint probing alone. These are imported via a bidirectional converter between rdf-solve's mapping edge representation and SeMRA's native `Mapping` objects, preserving evidence metadata.

All three sources produce edges in the same internal format (`MappingEdge`: source class, target class, predicate, source and target dataset and endpoint), enabling uniform downstream processing.

The aggregated edges are then generalised to type-level correspondences via the class-index lookup and type-pair derivation described below. Inference over the combined mapping graph subsequently propagates new links by applying transitivity, inversion, and optionally generalisation, using the SeMRA inference engine [?] (Section **??**).

All mappings carry provenance metadata including evidence chains and justification predicates from the SSSOM and semapv vocabularies.

Type lookup. For every entity e in the mapping set M, the pipeline first expands e to all registered URI prefix forms for the same identifier using bioregistry [?] (e.g.

`identifiers.org`, `purl.obolibrary.org`, and OBO PURL forms for the same entity), ensuring that the query covers every representation that may appear in G. Entities are then looked up in batches via a single SPARQL query that retrieves, for each entity, the named graphs in which it appears and the `rdf:type` values asserted in each graph:

```
SELECT ?e ?g ?type WHERE {
  VALUES ?e { <iri_1> <iri_2> ... }
  GRAPH ?g { ?e a ?type }
}
```

Results from all URI forms for the same entity are merged back under its canonical identifier. The resulting lookup table is optionally cached so that repeated runs do not re-query the endpoint.

Type-pair derivation. Algorithm **??** formalises the derivation step.

Algorithm 2 Type-level mapping derivation from instance evidence

Require: mapping set M (instance edges (s, p, o)); type lookup $\mathcal{I} : e \mapsto \{\text{graph} \mapsto [\text{type}, \ldots]\}$; thresholds $n_{\min}$, τ
Ensure: type-pair list $\mathcal{P}$, sorted by confidence

```
A ← empty counter                                  ▷ key: (t_s, t_o); value: supporting instance count
for each (s, p, o) ∈ M do
    for each type t_s of s in I do
        for each type t_o of o in I do
            A[t_s, t_o] += 1
        end for
    end for
end for
P ← { (t_s, t_o, A[t_s, t_o]) | A[t_s, t_o] ≥ n_min and conf(A[t_s, t_o]) ≥ τ }
return P                                           ▷ conf(n) = min(1, log_2(1 + n)/10). Placeholder, see Section ??
```

The output type pairs with supporting instance counts and confidence scores are serialised as a mapping artefact with full provenance metadata alongside a run report recording lookup cost, the fraction of entities successfully resolved, and the distribution of pair counts after filtering. An optional enrichment step annotates the original instance mapping files in place with the resolved type information, allowing downstream inspection of which ontology class types were recovered for each entity.

Inference expansion. After type-pair derivation, inference over the combined mapping graph propagates new links using the SeMRA inference engine [**?**]. Three operations are applied in sequence: (i) *inversion*: every mapping is made symmetric by generating its reverse; (ii) *transitivity*: transitive chains are inferred up to a configurable maximum chain length (default 3); (iii) *generalisation* (optional): broader/narrower relationships

are propagated to derive additional correspondences. After inference, duplicate mappings are deduplicated via evidence assembly, merging the evidence chains from all sources that produced the same mapping. The output is a single consolidated mapping artefact with provenance metadata recording which inference operations were applied and which source files contributed.

2.4 Integration of SPARQL examples

2.5 Query generator

2.6 Standard serializations

Extracted schemas are serialised in several standard formats. JSON-LD documents include a provenance block recording the extraction strategy, endpoint URL, generation timestamp, and pattern count, making each schema self-describing and ready for downstream use without requiring external metadata. VoID-ext Turtle files describe class and property partitions following the extended VoID vocabulary [**?**], generated directly from the internal schema model. SHACL shapes are produced through a two-step conversion: the JSON-LD schema is first translated to a LinkML schema, which is then passed through the LinkML `ShaclGenerator`; the resulting shapes can be applied directly for data validation or repair. JSON-LD, SHACL, and VoID artefacts are compatible with standard RDF tooling including rdflib and Apache Jena. The LinkML intermediate additionally enables downstream exports such as Pydantic Python APIs and SQL tables.

Provenance blocks additionally record the tool version, Python version, duration, machine fingerprint, and, where supplied, the ORCID-bearing author list of the person or workflow that triggered the extraction. This allows mining runs to be cited and traced to specific pipeline invocations. All schema and VoID artifacts are assigned canonical URIs under a shared base, so cross-referencing between a report, its VoID catalog, and its LinkML schema is machine-readable without extra configuration.

The web backend persists all artefact types (mined schemas, VoID catalogs, LinkML schemas, and mining reports) in a structured database, and exposes them through dedicated API endpoints (`/api/reports`, `/api/void`, `/api/linkml`, etc.). This decouples downstream consumers from the filesystem layout of the pipeline output and allows artefacts to be queried, filtered by dataset or strategy, and downloaded in their native formats (Turtle, YAML) directly via HTTP.

2.7 Implementation and reproducibility

The methodology is packaged as two Docker images.

The *pipeline script* bundles QLever, the format converters, and the extraction and mapping tools; it is the environment used for all experiments reported in this paper. This pipeline drives the complete workflow: running remote VoID discovery, optionally mining remote endpoints directly, generating per-source QLever configurations, downloading and indexing each dataset, mining schemas locally, executing the mapping and inference stages, and collecting results. The mapping stages comprise six dedicated pipeline steps: seeding SSSOM mapping bundles; importing SeMRA identifier mappings; running instance-based URI probing against each data source; restarting all locally indexed datasets as a unified LSLOD endpoint; building a class index by expanding every entity IRI to all registered bioregistry URI forms and querying the LSLOD endpoint for `rdf:type` assertions per named graph; and deriving type-pair correspondences with confidence scores from the aggregated instance evidence (Algorithm **??**). An optional enrichment pass annotates each source mapping file in place with the resolved type information. A separate inference step then applies transitivity, inversion, and generalisation over the combined mapping graph.

A separate *web application container* packages a Flask backend with a SQLite store for the extracted schemas and mappings, together with a TypeScript frontend for interactive exploration: dataset and mapping-source selection, diagram-based schema visualization, a SHACL pattern browser and editor, and an IRI resolver-based query generator. The web container is not required to reproduce the results; it serves as an interface for navigating the outputs produced by the pipeline.

Local mining Remote SPARQL mining depends on endpoint availability, rate limits, and timeout behavior, all of which vary across engines and time. To decouple extraction from these constraints, rdf-solve supports a local mining mode that operates on downloaded data dumps.

For each source that publishes downloadable data (N-Quads, RDF/XML, or compressed archives), the pipeline generates a QLever configuration file containing the appropriate download and conversion commands. QLever [**?**] indexes the data locally and exposes a SPARQL endpoint, against which the same mining algorithm (Algorithm **??**) is applied.

Because the local engine runs without network overhead, timeouts, or rate limits, mining completes faster and with fewer fallback activations.

The absence of network constraints makes a qualitatively different execution mode

practical. For local QLever instances the pipeline issues three *single unpaginated* `SELECT DISTINCT` queries (one each for typed-object, literal, and untyped-URI patterns) that attempt to retrieve the complete schema from the full index without pagination or batching. When all three succeed, which is reliably the case for all but the very largest datasets, no fallback logic is needed and the schema is complete in three round trips.

This execution path is *only* available for local mining; remote endpoints always begin directly with the fallback chain, since the endpoint's timeout and rate-limit behaviour is unknown.

This local mode complements remote mining. Sources that offer both a SPARQL endpoint and data dumps can be mined both ways, providing a cross-validation baseline for extraction completeness. Sources with unstable or slow endpoints can be mined locally without depending on their availability, and some sources with unmaintained endpoints need to be mined locally.

The pipeline supports executing the default and the untyped-as-classes extraction strategies consecutively for each source without re-downloading or re-indexing. The resulting pair of schema variants for each dataset allows downstream analysis to assess how much additional structure is recovered when untyped URI patterns are promoted to classes, and whether those candidate classes correspond to known ontology terms.

3 Results

3.1 Mining performance and reliability

We applied the mining algorithm (Section **??**) to N SPARQL endpoints.

Fallback utilization. For sources mined locally via QLever, the one-shot strategy succeeded in all cases tested, with no fallback activations required. Remote endpoint mining activated the fallback chain for a subset of sources where the direct query timed out or returned partial results; the multi-level chain (bisection, pagination, predicate-first) recovered the patterns in each case.

Pipeline step timing. Figure **??** shows the duration of each pipeline step recorded during a full run. Total runtime comprises, depending on whether extraction was local or remote:

- QLever indexing, one-shot schema extraction, mapping import, inference expansion, and graph construction (local); or
- remote endpoint mining, mapping import, inference expansion, and graph construction (remote).

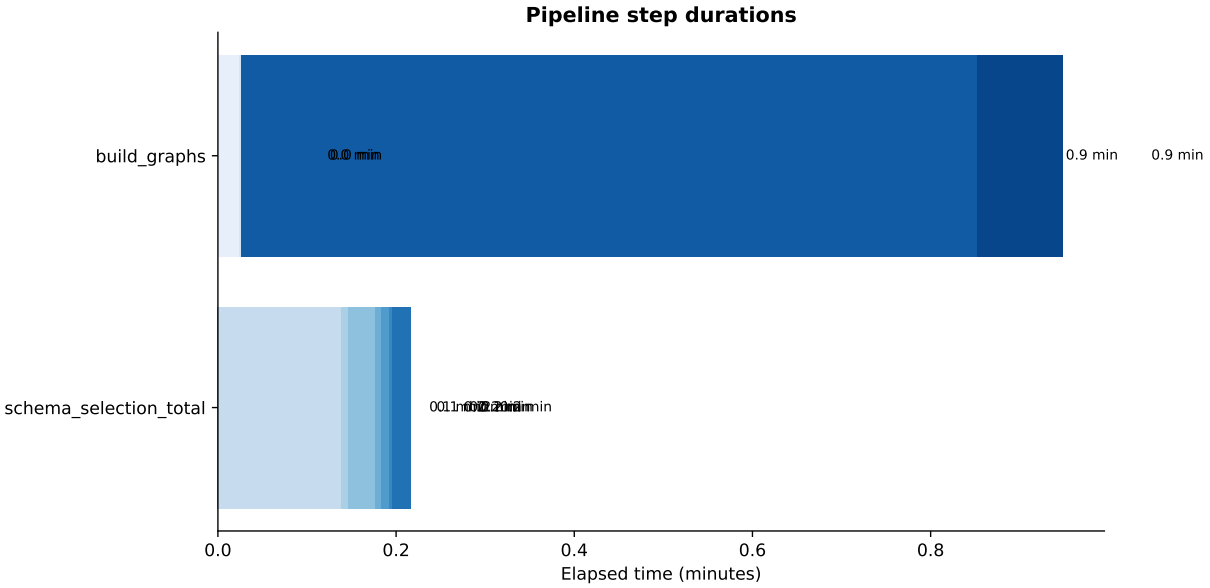

Figure 2: Duration of each pipeline step. (...).

3.2 Pattern coverage and structural profile

Pattern-type distribution.

Figure placeholder: `figures/fig_schema_profile.pdf`

Figure 3: Distribution of pattern and class counts across mined schemas (...).

Class coverage.

Property reuse across classes.

3.3 Mapping inventory and inference expansion

Raw mappings. Instance-level mappings from URI probing and SSSOM imports provide the input to the type-pair derivation step. Each source is serialized as a per-prefix file; when a prefix has been covered by multiple sources, the files are merged before processing. For each entity in the combined mapping set, its `rdf:type` values across all named graphs are retrieved from the unified endpoint, with bioregistry-based URI expansion to cover all registered identifier forms. Algorithm **??** then counts how many instance edges support each ontological type pair; run reports record the fraction of entities successfully resolved and the distribution of pair counts after filtering.

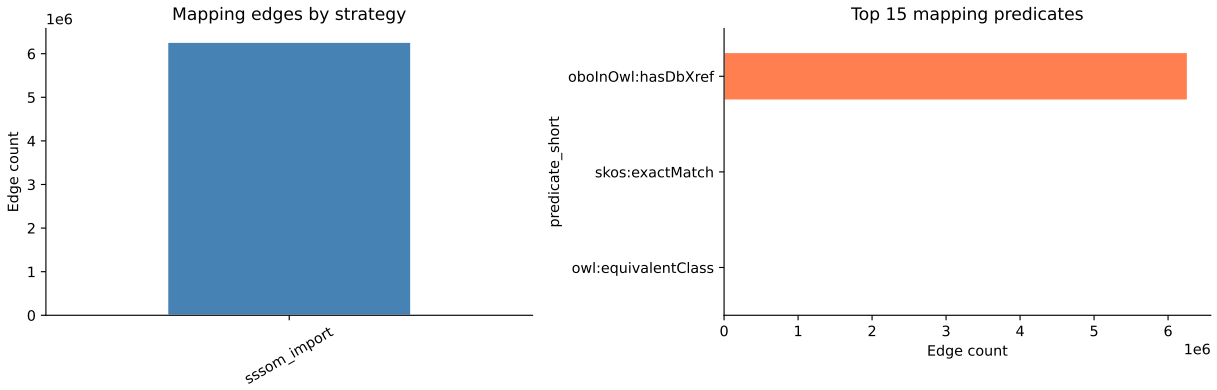

Figure 4: Distribution of mapping edges by strategy (left) and predicate (right).

Imported mappings. SeMRA-imported mapping sets provide instance-level edges that feed directly into Algorithm **??** alongside SSSOM imports and URI-probing results. Each imported edge contributes one vote toward the type-pair count for the ontological types of its subject and object, so curated and mined evidence are aggregated on the same confidence scale.

Inference expansion. Inference over the combined mapping graph (transitivity, inversion, and generalisation) produces additional type-level links not present in either the instance-based or imported layers.

3.4 Cross-endpoint connectivity graph

Graph construction.

Connectivity metrics. Table ?? summarises the connectivity of the three graph variants. Each row corresponds to one variant; columns report node count, edge count, number of connected components, largest-component size, diameter, graph density, number of Louvain communities, and modularity.

Table 2: Summary of dataset-level graph properties and Louvain community structure.

Graph variant	$\lvert V\rvert$	$\lvert E\rvert$	k	$\lvert V_{\text{lcc}}\rvert$	d	δ	c	Q	$\lvert V_{\text{lcc}}\rvert$ comm.
G_{schema}	88	869	28	61	4	0.227	3	0.544	42
G_{raw}	88	869	28	61	4	0.227	–	–	–
G_{inferred}	88	869	28	61	4	0.227	3	0.544	42

$\lvert V\rvert$: nodes; $\lvert E\rvert$: edges; k: connected components; $\lvert V_{\text{lcc}}\rvert$: largest component nodes; d: diameter; δ: density; c: Louvain communities; Q: modularity; comm.: nodes in largest community.

Hidden bridges revealed by mappings. Dataset pairs that are reachable in G_{inferred} but have no direct edge in G_{raw} represent *hidden bridges*: connections that exist only because a transitive or inverse mapping chain links the two datasets indirectly. Figure ?? visualises how inference expansion reshapes the community structure relative to Figure ??.

Hub ontology analysis. Datasets with the highest betweenness centrality in G_{inferred} act as integration hubs, bridging otherwise disconnected components. The heatmap in Figure ?? shows the log-scaled edge weights for the top datasets by betweenness centrality, highlighting the asymmetry between schema-level and inference-expanded connectivity.

Connectivity visualization.

3.5 Case study: cross-endpoint traversal

3.6 Comparison with prior work

Vocabulary reuse re-estimation.

Connectivity vs community detection.

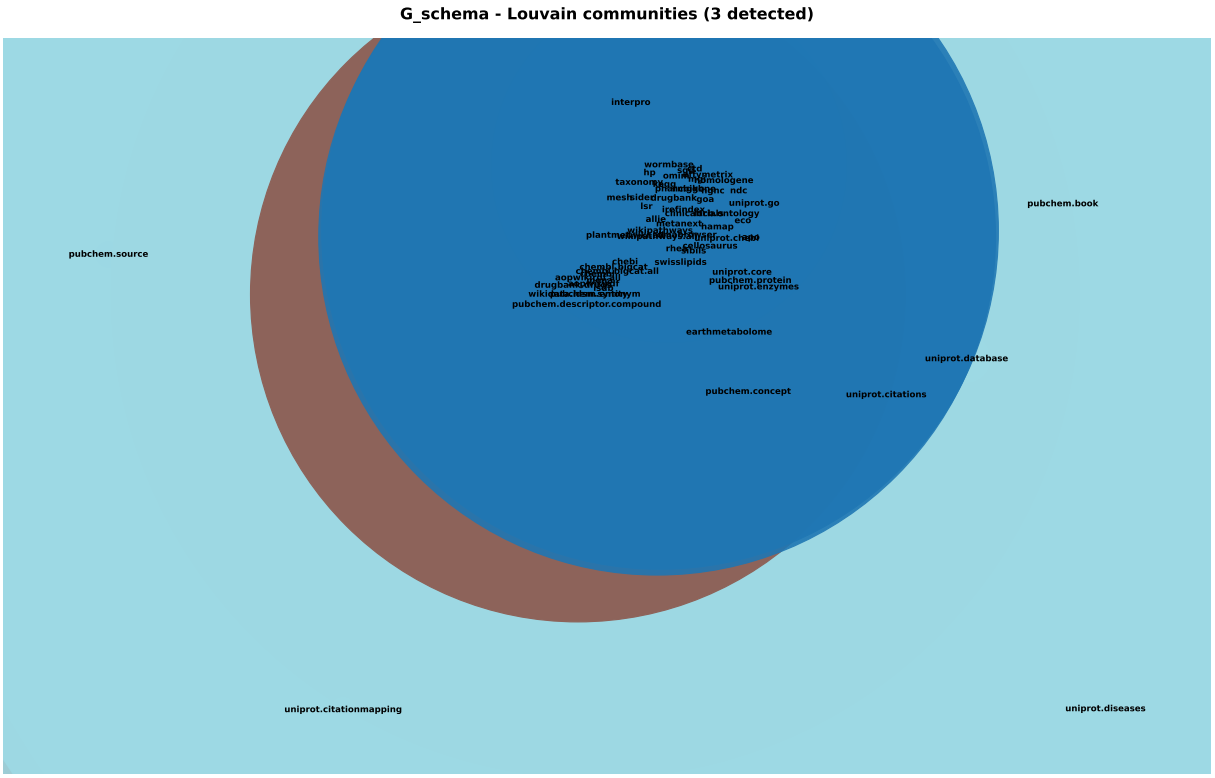

Figure 5: Community structure of G_{schema}. Nodes = datasets; edges = shared typed-object class pairs; colours = Louvain communities.

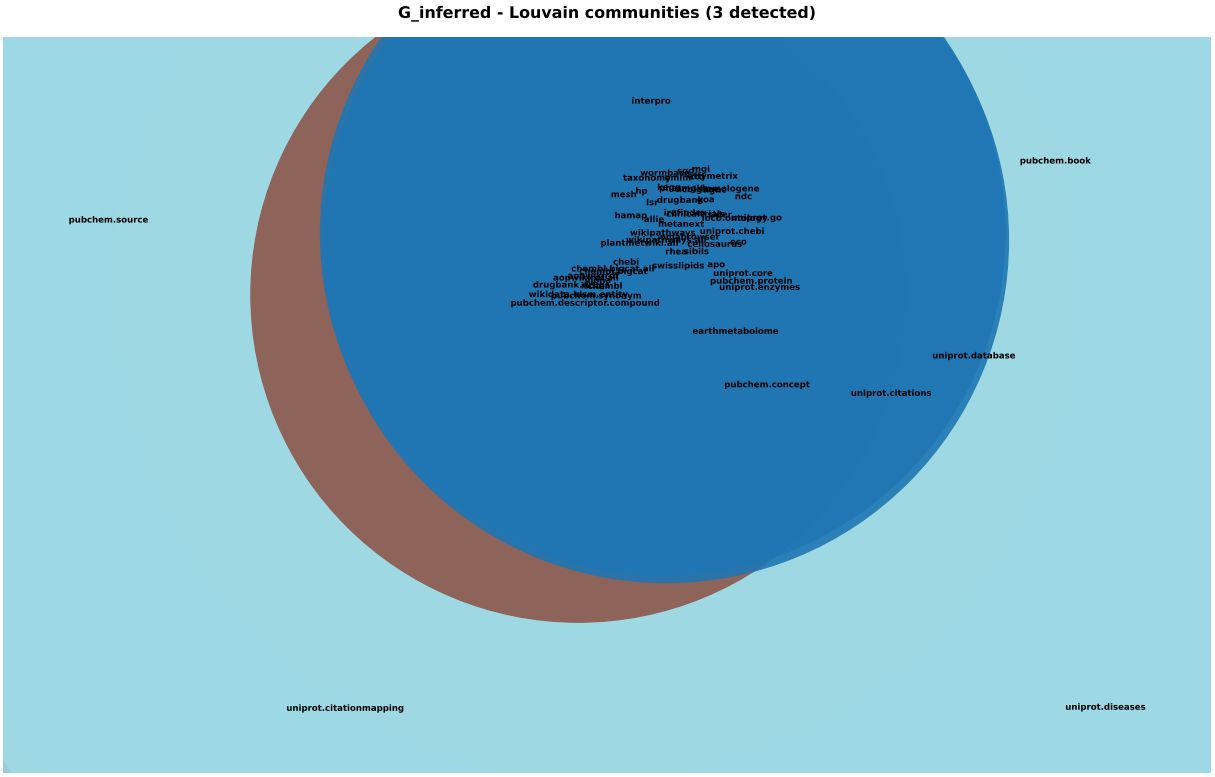

Figure 6: Community structure of G_{inferred} after all mapping layers and inference expansion.

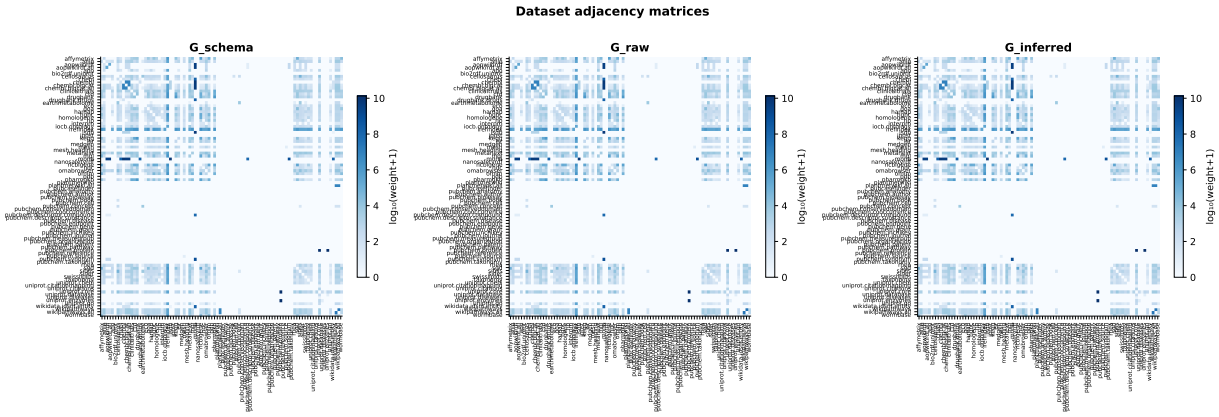

Figure 7: Log-scaled cross-dataset mapping weight for the top datasets by betweenness centrality in G_{inferred}.

3.7 Comparison with non-SPARQL-based mining methods

3.8 Schema validation and refinement

The methodology also exposes which classes are untyped in the RDF graphs (i.e. URI objects that lack explicit `rdf:type` assertions), which could be used iteratively to identify where new ontology classes need to be defined in order for the endpoint to be fully self-documented.

3.9 Traversal query generation and execution

4 Discussion

4.1 Positioning relative to SPARQL engine advances

Recent engineering work on QLever addresses the *engine* side of the scalability problem: block-based storage, precomputed predicate patterns, and autocompletion over billion-triple graphs [**?**, **?**]. GRASP shows that an LLM agent can generate correct SPARQL for a single, well-maintained endpoint by probing its schema interactively [**?**]. These are substantial technical advances.

Cross-endpoint against single-endpoint. Federated queries or dataset selection across the LSLOD cloud require prior knowledge of which endpoints share compatible schemas. The connectivity graph produced here provides that prior, though its quality depends directly on the completeness of the mined schemas and the precision of the imported mapping sets, both of which carry known limitations (Section **??**).

Persistent, citable artefacts versus on-the-fly discovery. GRASP discovers schema information at query time. The artefacts produced here (VoID catalogs, SHACL shapes, LinkML schemas) are static snapshots with provenance metadata.

Open science outputs for under-resourced endpoints. The resources that QLever autocompletion and LLM approaches perform best on tend to be large, consistently available, well-maintained endpoints (Wikidata, UniProt). Much of the LSLOD cloud is: small-to-medium databases maintained by small groups, with intermittent support, no VoID-ext

or other schema descriptions, and no budget for infrastructure upgrades. Publishing extracted schemas as linked data lowers the barrier for tools and workflows that cannot query the source directly. Whether this constitutes a meaningful contribution depends on adoption.

[?] concludes that the LSLOD cloud "is not actually as densely connected as is often indicated", with vocabulary reuse at under 20 % and many sources standing as isolated RDF conversions. Their community detection identified groups of semantically similar schema elements across sources using word-embedding similarity, revealing latent overlap invisible at the URI level. SynopsViz [?] and Sparklis [?] address exploration and querying of individual sources but do not bridge the gap across endpoints. LODsyndesis [?] provides connectivity metrics across hundreds of datasets but operates on pre-computed indexes rather than live endpoint mining.

We materialize the connectivity that prior analyses only detected statistically. The connectivity graph (Section **??**) identifies endpoint pairs that can be concretely joined via shared ontology classes and inferred mappings, answering “which endpoints can be federated?” rather than “which schemas are similar?”

Curated mapping sets imported further cover cases where namespace alignment alone is insufficient.

4.2 Comparison with LLM-driven query orchestration

TogoMCP [?] demonstrates that structured schema context is the decisive factor for accurate SPARQL generation by an LLM, finding it more valuable than elaborate behavioural prompting. Their MIE files, consisting of YAML documents bundling ShEx shapes, sample triples, and basic statistics per database, serve a role analogous to rdf-solve’s mined schemas: both supply the structured descriptions that endpoints themselves rarely publish. The two systems are, however, complementary rather than competing, differing in scope, automation, and downstream use.

Automated extraction versus manual curation. Each MIE file is curated semi-automatically per database with manual review [?], a process that scales well for the approximately 70 resources in the RDF Portal but becomes a bottleneck when extended to the full LSLOD cloud. rdf-solve generates schema descriptions automatically through its mining algorithm (Algorithm **??**), with the multi-level fallback chain and one-shot local mining recovering patterns from hundreds of endpoints without per-source engineer-

ing. The trade-off is precision: manually reviewed ShEx shapes may be more accurate for individual endpoints, while mined schemas trade curation effort for breadth of coverage. Because rdf-solve serialises its output as JSON-LD, LinkML, and SHACL, the mined schemas could in principle be reformatted as MIE files, supplying TogoMCP-style LLM context for databases that lack manually curated descriptors.

Cross-endpoint mappings and connectivity. TogoMCP operates on individual endpoints and bridges databases via sequential identifier conversion through TogoID, without native cross-endpoint JOIN support [**?**]. rdf-solve's mapping layer aggregates instance-based URI probing, curated SSSOM mapping sets, and SeMRA identifier mappings into a unified type-level correspondence graph (Section **??**), with inference expansion via transitivity, inversion, and generalisation. The resulting connectivity graph directly answers which endpoint pairs can be concretely joined, a capability that MIE-based approaches do not address.

Entity resolution. TogoMCP decouples entity resolution from SPARQL generation by delegating identifier lookup to external REST APIs (UniProt, ChEMBL, TogoID), a design that prevents the LLM from fabricating identifiers within queries. rdf-solve addresses a complementary aspect of the same problem: its bioregistry-based URI expansion (Section **??**) resolves every entity IRI to all registered URI forms before querying the LSLOD endpoint for `rdf:type` assertions, ensuring that identifier notation heterogeneity does not cause silent misses in the class index.

MCP integration as future work. Exposing rdf-solve's mined schemas, mappings, and connectivity graph as MCP tool resources would allow LLM agents to consume these artefacts at query time in the same protocol-driven pattern demonstrated by TogoMCP. This would combine rdf-solve's automated, cross-endpoint schema descriptions with the interactive query generation workflow that TogoMCP has shown to be effective, without requiring per-database manual curation of context files.

4.3 Provenance and reproducibility

Each mining run produces a self-contained report that records the tool version, extraction strategy, endpoint URL, graph scope, duration, warnings found, and the identity of the person or automated workflow that triggered it. These reports are assigned canonical URIs

and cross-referenced with their corresponding VoID catalog, LinkML schema, and SHACL shapes, so the full provenance chain of any artefact can be reconstructed without consulting external logs.

Documenting the extraction strategy, including whether counts were collected and whether the untyped-as-classes variant was applied, means that two schemas generated from the same endpoint under different assumptions are distinguishable in the record. This is a prerequisite for any meaningful longitudinal comparison or method benchmarking: conclusions about schema evolution or strategy differences require that the methodological parameters of each run are unambiguously captured alongside the results.

4.4 Scalability and centralization

Papadaki et al. [**?**] highlighted the difficulty of achieving efficiency for complex aggregate queries at scale.

Kamdar and Musen [**?**] argued for centralized top-down approaches to monitor accessibility and enforce vocabulary reuse standards.

rdf-solve's multi-level fallback chain addresses the scalability concern at the extraction level for external endpoints, recovering patterns for queries that would otherwise fail under single-shot extraction. The local mining pipeline (Section **??**) further removes the dependency on remote endpoint availability entirely: downloading data dumps and indexing them with QLever eliminates network timeouts, rate limits, and engine-specific quirks from the extraction path. The containerized web application and the batch-seeding scripts provide a reusable infrastructure for periodic re-mining, moving toward the centralized monitoring advocated by Kamdar and Musen [**?**].

4.5 Mining robustness and the fallback contribution

The multi-level fallback chain (batch bisection, pagination with adaptive page sizing, predicate-first decomposition) is an engineering contribution that enables all downstream analyses for sources lacking RDF dumps.

Prior remote endpoint miners, including the tool used by [**?**] and Loupe [**?**], issue single-shot queries per class and silently skip classes on timeout.

The separation of pattern discovery from counting further ensures that a count failure does not cascade into a pattern loss, a design choice validated by the count-failure rate analysis.

4.6 Resulting LSLOD cloud

Schemas

Mapping layer

4.7 Ontology reuse and matching

4.8 Mapping quality and evidence chains

The mapping layer's quality is bound by its upstream sources. Instance-level mappings from URI probing, SSSOM imports, and SeMRA imports each inherit the precision and recall of their respective inputs Hoyt et al. [**?**]. Inference propagation (transitivity, inversion, generalisation) amplifies both signal and noise: transitive chains of length ≥ 3 carry multiplicative uncertainty. The evidence-chain metadata preserved in the mapping JSON-LD documents these provenance paths, but downstream consumers must decide which evidence depth to trust.

The current confidence function (Algorithm **??**) is a placeholder calibrated only to raw instance count: $\mathrm{conf}(n) = \min(1, \log_2(1+n)/10)$. A probabilistic formulation would instead estimate the posterior probability $P(t_s \equiv t_o \mid \mathcal{E})$ that a type pair represents a genuine ontological correspondence given evidence set $\mathcal{E}$. One natural approach, informed by the SeMRA framework's predicate semantics, would treat each instance edge as a Bernoulli trial: if the true correspondence probability is θ, the likelihood of observing n supporting edges out of N total entity pairs touching those types is $\mathrm{Bin}(n; N, \theta)$, with a Beta prior on θ updated per predicate type and evidence justification. This formulation would additionally allow a consistency check across multiple independent mapping sources: a type pair supported by sources that systematically disagree on the predicate (e.g. some asserting exact match, others only broad match) would receive a lower posterior than a pair unanimously supported. These refinements require calibration data and are left for future work.

The type lookup is limited to `rdf:type` assertions that are materialised and visible under the query scope in G. Entities whose type membership is only entailed, or is asserted in a named graph not covered by the lookup query, will not be resolved and their edges will be silently dropped during derivation. Run reports record the fraction of edges skipped for this reason, enabling targeted diagnosis.

4.9 Limitations of schema mining

Graph scope and invisible types. Phase 1 enumerates only types asserted via `rdf:type` and visible under the chosen graph scope; inferred or unmaterialized types are silently absent from C. Graph selection is therefore a critical configuration choice. The whole-endpoint mining variant (no named-graph restriction) with post-mining service-namespace filtering addresses this partially: it recovers patterns that would be missed under a strict graph scope, at the cost of including infrastructure artefacts that must be stripped.

Remote mining. The remote mining step depends on endpoint availability, rate limits, and engine-specific behavior, and is inherently liable to data incompleteness. Failure risk for Q_{to} grows with instance count and object cardinality, so the output may be biased toward small, well-indexed classes. When all fallback strategies fail, a batch contributes $\emptyset$ to S, indistinguishable from a class with genuinely no patterns of that type. Failures are logged but cannot be encoded in S itself.

Different SPARQL engines (Virtuoso, Blazegraph, Fuseki, QLever) exhibit different timeout behavior, error responses... The fallback chain mitigates but does not eliminate these differences.

4.10 Broader limitations and future work

Property-level mappings. The current mapping layer operates at the class level: it links ontological types across datasets but does not align individual predicates. Property-level mappings would enable predicate relaxation, e.g. mapping domain-specific relation ontology predicates to broader SIO predicates, which is a prerequisite for generating simplified property-graph exports and cross-dataset federated queries that abstract over predicate heterogeneity.

Validation feedback loops. Feeding validation results back to dataset publishers would close the loop between schema mining and data improvement Mihindukulasooriya et al. [**?**].

Downstream exports. LinkML intermediate representations open a path to SQL table generation, property-graph exports (Neo4j), and typed Python APIs (Pydantic), extending the utility of mined schemas beyond the RDF ecosystem.

Model Context Protocol integration. TogoMCP [?] has shown that exposing database context through the Model Context Protocol enables LLM agents to generate accurate SPARQL without manual prompt engineering. An MCP server layer over rdf-solve would expose mined schemas, mappings, and connectivity metadata as tool resources consumable by any MCP-compatible LLM client, effectively scaling the TogoMCP paradigm from manually curated MIE files to automatically generated schema artefacts across the full LSLOD cloud. Combined with the query composition module already present in rdf-solve, this would provide an end-to-end path from natural-language questions through schema-informed query generation to cross-endpoint traversal.

References

[1] Ziawasch Abedjan, Toni Gruetze, Anja Jentzsch, and Felix Naumann. Profiling and mining RDF data with ProLOD++. In *2014 IEEE 30th International Conference on Data Engineering*, pages 1198–1201, Chicago, IL, USA, March 2014. IEEE.

[2] Shqiponja Ahmetaj, Iovka Boneva, Jan Hidders, Katja Hose, Maxime Jakubowski, Jose Emilio Labra Gayo, Wim Martens, Fabio Mogavero, Filip Murlak, Cem Okulmus, Axel Polleres, Ognjen Savković, Mantas Šimkus, and Dominik Tomaszuk. Common Foundations for SHACL, ShEx, and PG-Schema. In *Proceedings of the ACM on Web Conference 2025*, pages 8–21, Sydney NSW Australia, April 2025. ACM.

[3] H Bast, J Kalmbach, C Ullinger, and R Textor-Falconi. Sparqloscope: A generic benchmark for the comprehensive and concise performance evaluation of SPARQL engines.

[4] Hannah Bast and Björn Buchhold. Qlever: A query engine for efficient sparql+ text search. In *Proceedings of the 2017 ACM on Conference on Information and Knowledge Management*, pages 647–656, 2017.

[5] Nikos Bikakis, Melina Skourla, and George Papastefanatos. rdf:SynopsViz - A Framework for Hierarchical Linked Data Visual Exploration and Analysis, 2014. Version Number: 2.

[6] Jerven Bolleman. void-generator, 2015.

[7] Emma Kathleen Carter. The "I" in FAIR- Translating from Interoperability in Principle to Interoperation in Practice.

[8] Sijin Cheng and Olaf Hartig. FedQPL: A Language for Logical Query Plans over Heterogeneous Federations of RDF Data Sources (Extended Version), October 2020. arXiv:2010.01190 [cs].

[9] Daniel Fernández-Álvarez and Jose Emilio Labra-Gayo. sheXer: A robust and versatile library for automatic extraction of RDF shapes. *SoftwareX*, 34:102592, June 2026.

[10] Daniel Fernández-Álvarez, Jose Emilio Labra-Gayo, and Daniel Gayo-Avello. Automatic extraction of shapes using sheXer. *Knowledge-Based Systems*, 238:107975, 2022.

[11] Daniel Fernández-Álvarez, Yasunori Yamamoto, Jose Emilio Labra Gayo, and Andra Waagmeester. Extracting shapes from large RDF data collections. In *Proceedings of the 15th Semantic Web Applications and Tools for Health Care and Life Sciences Conference*, 2024.

[12] Sébastien Ferré. Sparklis: An expressive query builder for SPARQL endpoints with guidance in natural language. *Semantic Web*, 8(3):405–418, December 2016.

[13] Nicola Guarino and Pierdaniele Giaretta. Ontologies and Knowledge Bases.

[14] Ali Hasnain, Qaiser Mehmood, Syeda Sana e Zainab, and Aidan Hogan. SPORTAL: Profiling the Content of Public SPARQL Endpoints. *International Journal on Semantic Web and Information Systems (IJSWIS)*, 12(3):134–163, July 2016.

[15] Charles Tapley Hoyt, Meghan Balk, Tiffany J. Callahan, Daniel Domingo-Fernández, Melissa A. Haendel, Harshad B. Hegde, Daniel S. Himmelstein, Klas Karis, John Kunze, Tiago Lubiana, Nicolas Matentzoglu, Julie McMurry, Sierra Moxon, Christopher J. Mungall, Adriano Rutz, Deepak R. Unni, Egon Willighagen, Donald Winston, and Benjamin M. Gyori. Unifying the identification of biomedical entities with the Bioregistry. *Scientific Data*, 9(1):714, November 2022.

[16] Charles Tapley Hoyt, Klas Karis, and Benjamin M Gyori. Assembly and reasoning over semantic mappings at scale for biomedical data integration. *Bioinformatics*, 41(11):btaf542, November 2025.

[17] Maulik R. Kamdar and Mark A. Musen. An empirical meta-analysis of the life sciences linked open data on the web. *Scientific Data*, 8(1):24, January 2021.

[18] Shuichi Kawashima, Toshiaki Katayama, Hideki Hatanaka, Tatsuya Kushida, and Toshihisa Takagi. NBDC RDF portal: a comprehensive repository for semantic data in life sciences. *Database*, 2018, January 2018.

[19] Akira R Kinjo, Yasunori Yamamoto, Samuel Bustamante-Larriet, Jose-Emilio Labra-Gayo, and Takatomo Fujisawa. TogoMCP: Natural Language Querying of Life-Science Knowledge Graphs via Schema-Guided LLMs and the Model Context Protocol.

[20] Pierre Maillot, Olivier Corby, Catherine Faron, Fabien Gandon, and Franck Michel. IndeGx: A model and a framework for indexing RDF knowledge graphs with SPARQL-based test suits. *Journal of Web Semantics*, 76:100775, April 2023.

[21] Nandana Mihindukulasooriya, Marıa Poveda-Villalon, Raul Garcıa-Castro, and Asuncion Gomez-Perez. Loupe - An RDF Dataset Description Model for Expressing Vocabulary Usage Patterns.

[22] Nandana Mihindukulasooriya, Mohammad Rifat Ahmmad Rashid, Giuseppe Rizzo, Raúl García-Castro, Oscar Corcho, and Marco Torchiano. RDF shape induction using knowledge base profiling. In *Proceedings of the 33rd Annual ACM Symposium on Applied Computing*, pages 1952–1959, 2018.

[23] Maria-Evangelia Papadaki, Yannis Tzitzikas, and Michalis Mountantonakis. A Brief Survey of Methods for Analytics over RDF Knowledge Graphs. *Analytics*, 2(1):55–74, January 2023.

[24] Blerina Spahiu, Andrea Maurino, and Matteo Palmonari. Towards Improving the Quality of Knowledge Graphs with Data-driven Ontology Patterns and SHACL. In *ISWC*, pages 103–117, 2018.

[25] Blerina Spahiu, Matteo Palmonari, Renzo Arturo Alva Principe, and Anisa Rula. Understanding the structure of knowledge graphs with ABSTAT profiles. *Semantic Web*, 15(5):1519–1545, October 2024.

[26] Ruben Taelman and Ruben Verborgh. Link Traversal Query Processing Over Decentralized Environments with Structural Assumptions. In Terry R. Payne, Valentina Presutti, Guilin Qi, María Poveda-Villalón, Giorgos Stoilos, Laura Hollink, Zoi Kaoudi, Gong Cheng, and Juanzi Li, editors, *The Semantic Web – ISWC 2023*, volume 14265, pages 3–22. Springer Nature Switzerland, Cham, 2023. Series Title: Lecture Notes in Computer Science.

[27] Yael Tirlet, Jerven Bolleman, Emmanuelle Becker, Fabrice Legeai, and Olivier Dameron. Modular composition of SPARQL queries for focusing on what to look for rather than how to get it.

[28] Sebastian Walter and Hannah Bast. GRASP: Generic Reasoning And SPARQL Generation across Knowledge Graphs. 2025.